Does Call-Graph Fan-Out Predict Failure Impact in Microservices? A Preregistered Fault-Injection Study

Dakshrajsinh Tank and Badrinath N

***Abstract*—Ranking the services of a microservice application by their place in the call graph is a cheap way to choose which ones to harden, and a common intuition treats services with high fan-out as the riskiest. A failed service, however, mainly hurts the services that wait on it, which are its callers. This paper tests both views in a preregistered fault-injection campaign of 180 runs on two DeathStarBench applications. Every analysable service received a process kill and a 500 ms network delay in 5 repetitions each. Dependency graphs were rebuilt from traces, and latency and errors were charged to each service's own spans. On Social Network under kill faults, the number of transitive callers and the number of degraded callers correlated at $\rho = 0.785$, but none of the 44 confirmatory tests survived Benjamini-Hochberg correction (adjusted $p = 0.3451$), and a latency-fault replication agreed. Much of that correlation holds by construction, since ten of the 11 services degraded every caller they had. A fan-out-weighted centrality metric correlated negatively with degraded callers in every application and fault type and was distinguishable from zero in none, and a post hoc sweep of 231 weightings found no significant association. The clearest result concerns direction: no callee of the faulted service degraded in 179 of 180 runs. The study also shows that charging a trace's end-to-end latency to every service on its path flags unrelated services, and it releases data and scripts that regenerate every reported number.**

***Index Terms*—Distributed tracing, failure propagation, fault injection, graph centrality, microservices, preregistration.**

I. Introduction

A microservice application is assembled from many small services that talk to each other over the network and can be built, deployed and scaled one at a time [1], [2]. Teams adopt the style for that independence, and pay for it in operations. A systematic review of grey literature records the pains practitioners report alongside the gains [3], and performance engineering for these systems has its own list of open research problems [4]. Failure is near the top of that list. A study of hundreds of cloud service outages draws lessons about why such systems stop working [5], and an industrial survey of microservice systems reports that faults are slow to diagnose [6].

Operators would like to know in advance which services deserve hardening first. Injecting faults into every service answers that directly, but each answer costs a run per service and per fault type. A cheaper answer can be read from the dependency graph. Graph measures have long served as indicators of importance [7], and network measures computed on software dependency graphs have predicted which components accumulate defects [8]. Applied to a service graph, the familiar reading is that a service calling many others is a hub, and a hub is dangerous. This project started from that reading. Its original metric, hybrid criticality, scales out-degree centrality by fan-out, the number of services a service calls directly, on the argument that a service with many callees cuts many links when it fails.

There is a simple argument in the other direction. When a service fails, the services left waiting are the ones that call it; its callees simply receive less traffic. On that argument the number of services that degrade should follow a service's transitive callers, its ancestors in the graph, and a fan-out ranking should go most wrong on exactly the services it rates highest. The composition service of the Social Network benchmark shows the clash in one place. It calls 7 services directly, and the fan-out metric ranks it first of the 11 services analysed. It has a single caller, so a ranking by callers puts it last.

This paper tests the caller-based prediction against measurement, with the fan-out metric kept as a comparator, in a preregistered [9] fault-injection campaign on two applications of the DeathStarBench suite [10]. The caller-based hypothesis was formed after a pilot study and locked, together with every analysis choice, before any campaign data existed. All 180 planned runs completed. Each analysable service received a process kill and an injected network delay, 5 times each. Dependency graphs were rebuilt from traces, latency was measured on the spans each service emits itself, and every degraded service was classified by its graph relation to the faulted one, so that degraded callers and degraded callees were counted separately.

The preregistered hypothesis was not confirmed. On Social Network under kill faults, the caller count and the number of degraded callers correlated at $\rho = 0.785$, but none of the 44 tests in the confirmatory family survived correction for multiple testing, and a replication under latency faults gave the same answer. A large part of that correlation is definitional, because the outcome counts a subset of the predictor and most services degraded every caller they had. The fan-out metric fared no better: its correlation with degraded callers was negative for both applications and both fault types, and in no case distinguishable from zero. The plainest result needs no test. Degradation reached callers and almost never callees: no descendant of the faulted service degraded in 179 of 180 runs.

D. Tank and Badrinath N are with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India.

Corresponding author: [TO FILL: corresponding author name and email].

The paper makes four contributions:

- A preregistered test of whether caller count, or a fan-out-weighted centrality metric, tracks measured failure impact in two benchmark applications, reported with its null result and with the ceiling that the design placed on any correlation.
- Evidence from 180 runs under two fault types that degradation travels to the callers of a failed service and almost never to its callees.
- A measurement pitfall in trace-based latency attribution, in which a request's end-to-end latency is charged to every service on its path, together with its effect in a pilot and the per-service attribution that removes it.
- An artifact in which a script regenerates every number in the paper from the logged campaign data.

Section II reviews related work and Section III states the problem and the hypotheses. Section IV describes the testbed and the method, Section V reports the results and Section VI interprets them. Section VII discusses threats to validity and Section VIII concludes.

II. Related Work

A. Graph Measures and Network Failures

Freeman's account of centrality separated several ideas of what makes a node central, some resting on a node's direct ties and others on the shortest paths that pass through it, and gave them normalised forms that remain standard [7]. Path-based measures became practical on large graphs once Brandes published a fast algorithm for betweenness [11]. The graph library used in this study, NetworkX 3.6.1 [12], implements that algorithm.

Network science has studied how graphs respond to the loss of nodes. Albert, Jeong and Barabási examined the tolerance of complex networks to errors and to attacks [13], and Motter and Lai examined attacks on complex networks that work through cascades [14]. In a running microservice system a failed service stays in the call graph, and its effect appears as latency and errors in the services around it, which have to be measured on the system itself.

Software engineering adopted graph measures early. Zimmermann and Nagappan computed network measures on the dependency graph of a large software system and found that they helped predict which components would contain defects [8], and Ma and colleagues built a service dependency graph to analyse and test microservice systems [15]. The present study shares their premise that a component's position in a dependency graph says something about its risk. Its outcome is different: how far a run-time failure spreads under load, which depends on the call pattern of the running system and not on the code alone.

B. Fault Injection and Chaos Engineering

Injecting faults into a running system is now an established way to learn how it behaves under stress. Basiri and colleagues describe chaos engineering as practised at Netflix, where controlled experiments on production systems build confidence that they tolerate failure [16], and Basiri and colleagues later described automating chaos experiments in production [17]. For microservices, Gremlin offered systematic resilience testing by injecting faults between services [18], and Meiklejohn and colleagues brought fault injection testing down to the level of individual services [19]. Lineage-driven fault injection reasons backwards from how a correct result was produced to choose the faults most likely to expose a bug [20]. Not every failure is a crash. Huang and colleagues describe gray failure, in which a component keeps running while working badly, as a central weakness of cloud-scale systems [21]; an injected network delay is one controlled form of such a partial failure.

These tools search for bugs or confirm that fallbacks work. This study uses fault injection for a narrower purpose. Two fault types are injected into each service in turn in a fixed, balanced design, and the counts of degraded services become the outcome against which a structural prediction is tested. The faults serve as a measuring instrument.

C. Call Graphs from Traces

Distributed tracing makes it possible to recover the service graph from what the system actually does. Dapper set out the model most tracing systems still follow: a request carries a trace context across service boundaries, each unit of work becomes a span with a parent, and a request becomes a tree of spans [22]. A parent span in one service with a child span in another gives a caller-to-callee edge, which is how the graphs in this study were built. Graphs recovered this way depend on the traffic that produced them. Luo and colleagues found the call graphs in Alibaba's production traces to be highly dynamic, changing from one request to the next [23]. The benchmark graphs here are small and their workload is fixed, but the same dependence on traffic reaches the predictor (Section VII-C).

D. Failure Diagnosis

A large body of work starts from an anomaly that is already under way and tries to find its source. Pinpoint addressed problem determination in large, dynamic Internet services [24]. For microservices, Microscope locates performance problems with causal graphs [25], MicroRCA ranks candidate root causes on a graph of the running system [26], and Seer applies large-scale data analysis to performance debugging in cloud microservices [27]. An industrial survey documents how hard such diagnosis is in practice and contributes a benchmark with injected faults [6], and Soldani and Brogi survey anomaly detection and root-cause analysis for service-based cloud applications [28]. These methods use the graph together with live metrics after a problem has begun. The question here comes earlier: whether graph structure alone, before any fault, indicates which services will do the most damage when they fail. The direction of propagation reported in Section V-F may interest diagnosis work, although no diagnosis method was studied.

E. Preregistration and Multiple Testing

Preregistration separates the analyses planned before data are seen from those devised afterwards, so that a reader can tell a test of a prediction from a search through the data [9]. With 44 tests in each family, a correction for multiple testing was unavoidable. The false discovery rate was controlled with the Benjamini-Hochberg procedure [29], which bounds the expected share of false positives among reported discoveries and gives up less power than a family-wise correction.

F. Research Gap

Graph measures have been used to rank software components by defect risk [8], network studies examine tolerance to errors, attacks and cascades [13], [14], fault-injection tools test whether a system tolerates failure [16], [18], [19], and diagnosis methods use the graph once an anomaly is under way [25], [26]. To the authors' knowledge, none of the works cited here reports a test, fixed in advance, of whether a graph-derived ranking of services anticipates failure propagation measured in a controlled campaign. This paper reports such a test, including a result that came out null.

III. Problem Statement and Hypotheses

A. Call Graph, Faults and Outcome

Let $G = (V, E)$ be the directed call graph of an application. V is the set of services, and an edge $(u, v) \in E$ means that u calls v under the traced workload. Writing $u \rightsquigarrow v$ for a directed path from u to v, the ancestors of a service are the services from which it can be reached and its descendants are the services it can reach:

$$\mathrm{anc}(v) = \{u \in V : u \rightsquigarrow v,\ u \neq v\}, \tag{1}$$

$$\mathrm{desc}(v) = \{u \in V : v \rightsquigarrow u,\ u \neq v\}. \tag{2}$$

A fault $f \in \{\mathrm{kill}, \mathrm{latency}\}$ applied to service v either terminates its process or delays its network traffic (Section IV-E). In repetition r the degraded set $D_r(v, f)$ holds the services other than v whose own latency or error rate crosses the degradation rule of Section IV-G during the fault window. The degraded set splits into three disjoint parts by graph relation to the faulted service:

$$D_r^A = D_r \cap \mathrm{anc}(v), \qquad D_r^D = D_r \cap \mathrm{desc}(v), \tag{3}$$

$$D_r^U = D_r \setminus \big(\mathrm{anc}(v) \cup \mathrm{desc}(v)\big). \tag{4}$$

The primary outcome of a service is the median over its repetitions of the number of degraded ancestors,

$$Y(v, f) = \mathrm{median}_r\, |D_r^A(v, f)|, \tag{5}$$

called the number of degraded callers below. The counts of degraded descendants and of degraded unrelated services are recorded separately and never added to Y, because a combined count would hide the direction in which a failure travels.

B. Predictors

The primary predictor is the caller count, the number of transitive callers $X(v) = |\mathrm{anc}(v)|$. The comparator of interest is the hybrid criticality metric of the original proposal,

$$\mathrm{HC}(v) = w_{\mathrm{in}} C_{\mathrm{in}}(v) + w_{\mathrm{out}} C_{\mathrm{out}}(v)\, \varphi(v) + w_{\mathrm{btw}} C_{\mathrm{btw}}(v), \tag{6}$$

$$\varphi(v) = d^+(v) \,/\, \max_u d^+(u), \tag{7}$$

where C_{in} and C_{out} are in-degree and out-degree centrality, C_{btw} is betweenness with directed normalisation, d^+ is out-degree, and the weights are $w_{\mathrm{in}} = 0.4$, $w_{\mathrm{out}} = 0.4$ and $w_{\mathrm{btw}} = 0.2$ (Section IV-D). The factor φ grows linearly with out-degree, so the metric rewards services that call many others. Ten further graph measures are kept as comparators. Each test pairs one predictor with one outcome across the services of one application, with the gateway left out (Section IV-C).

C. Hypotheses

The hypotheses are those of the locked preregistration, where the second carries the label H1b because the name H2 there belongs to an untested structural observation. H1, the primary hypothesis, states that the caller count predicts Y while out-degree, descendant count and the hybrid metric do not. It is tested as a positive Spearman rank correlation:

$$\text{H1:} \quad \rho(X, Y) > 0. \tag{8}$$

H2 is directional. It states that the fan-out premise points the wrong way, so that the hybrid metric is not positively associated with the outcome:

$$\text{H2:} \quad \rho(\mathrm{HC}, Y) \leq 0. \tag{9}$$

The null hypothesis of each test is that predictor and outcome have no monotone association. Every p-value is two-sided, comes from a permutation test and is corrected with the Benjamini-Hochberg procedure within its family (Section IV-H). Kill faults form the confirmatory family, and latency faults repeat every test as a separate replication family.

IV. Testbed and Methodology

Fig. 1 shows the pipeline in the order the campaign ran it. The first steps built the dependency graphs and the predictors and locked the analysis plan before any campaign data existed. The middle steps repeated in every run, and the last ones ran once the campaign was complete. Each box names the repository files that carry out the step.

A. Testbed

All experiments ran on one host, configured as in Table I. The application stacks, Jaeger and Pumba ran as containers inside the virtual machine of Docker Desktop's WSL2 backend, while the load generator and the campaign runner ran as host processes (Fig. 2). The runner stops every stack before the campaign and brings up one application at a time, so the two never shared the 7.598 GiB memory ceiling of the virtual machine. The Social Network stack has 27 containers and the Hotel Reservation stack 24. Traces were collected with Jaeger

TABLE I
TESTBED AND RUN PROTOCOL; HOST VALUES MARKED AS MEASURED WERE QUERIED AFTER THE CAMPAIGN

Item	Value	Source
Host OS	Microsoft Windows 11 Home Single Language (10.0.26300)	measured on 2026-10-20
Host CPU	12th Gen Intel(R) Core(TM) i7-1255U, 10 cores, 12 threads	measured on 2026-10-20
Host RAM (GB)	15.7	recorded 2026-09-24
Docker Desktop	4.91.0, WSL2 backend	version measured on 2026-10-20
Docker VM memory (GiB)	7.598 (WSL2 default)	recorded 2026-09-24
Docker client / Compose	29.8.0 / 5.5.1	client recorded 2026-10-18; Compose measured on 2026-10-20
Containers per stack	Social Network 27, Hotel Reservation 24	recorded 2026-09-24; one stack at a time
DeathStarBench commit	6ecb097	pinned upstream commit
Jaeger	1.62.0 all-in-one, in-memory storage	compose override file
Pumba	ghcr.io/alexei-led/pumba, untagged image	version not recorded
Python	3.13.2	repository README
Python libraries	NumPy 2.1.2, pandas 2.2.3, SciPy 1.14.1, statsmodels 0.14.6, NetworkX 3.6.1, matplotlib 3.9.2	repository README
Locust	2.46.3; 25 users, spawn rate 10 users/s, 0.5 to 1.5 s between requests	runner and Locust files
Request mix, Social Network	compose post 3, home timeline 5, user timeline 2	Locust task weights
Request mix, Hotel Reservation	search 6, reservation 2, user 2, recommendation 1	Locust task weights
Load before the fault (s)	60, also the baseline window	runner
Fault hold (s)	30	runner
Recovery probe (s)	poll 1, confirmation 5, lag 5, cap 180	runner
Cooldown (s)	20	runner

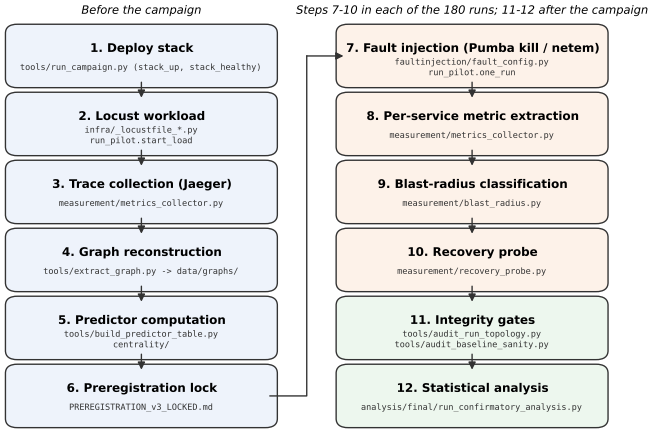


Fig. 1. Pipeline in execution order. Each box names the repository files that carry out the step; the steps from fault injection to the recovery probe repeat in every run.

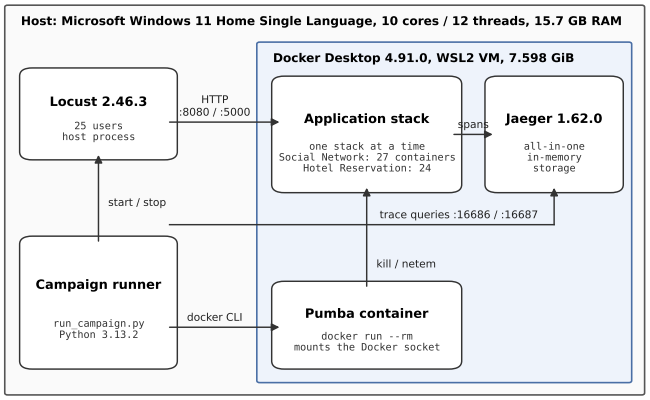


Fig. 2. Testbed deployment: the host, the Docker Desktop virtual machine with the application stack, Jaeger and Pumba containers, and the Locust load generator and campaign runner on the host.

1.62.0, one all-in-one instance per application with in-memory storage. The memory ceiling, host memory and container counts were recorded on 2026-09-24, before the campaign. The host operating system, CPU and Docker Desktop version were queried on 2026-10-20, after the campaign, and Table I marks them as measured.

B. Applications and Dependency Graphs

Both applications come from DeathStarBench [10], built from one pinned upstream commit, 6ecb097, so the code under test was identical in every run. Their shapes differ. Social Network is deep and fans out from a single composition service, while Hotel Reservation is shallow and close to a star around its gateway. That difference turned out to decide what each application could show (Section V-B).

The dependency graphs were taken from traces and not from documentation, because a diagram shows the calls a system can make while the hypotheses concern the calls it makes under the workload being perturbed. A directed edge runs from caller to callee wherever a span of one service is the parent of a span of another [22]. The Social Network graph came from 2,449 traces holding 32,847 spans, and the Hotel Reservation graph from 3,446 traces holding 31,155 spans. The graphs have 12 and 8 services, with 17 and 7 observed call edges (Figs. 3 and 4). Both were frozen as files, and every predictor is computed from those files.

The observed Hotel Reservation graph differs from the documented one. Of the 8 call edges implied by the published service tree, the traces recover 6. Two never appear under this workload, reservation→user; search→profile, and one edge the tree does not list appears, frontend→profile, which seems to

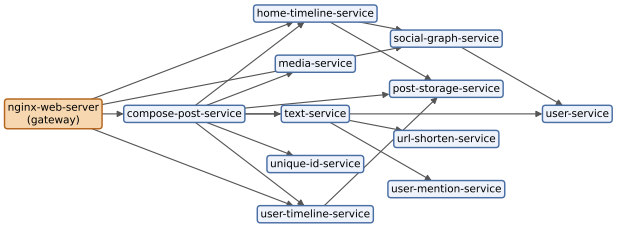


Fig. 3. Social Network dependency graph as observed in traces, with the gateway marked.

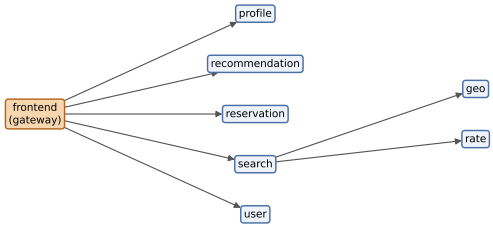


Fig. 4. Hotel Reservation dependency graph as observed in traces, with the gateway marked.

stand in for the missing search-to-profile call. The difference matters for one service: user has a single observed caller, frontend, where the documentation implies two. The observed graph is used, and the divergence is reported.

C. Analysis Set

Every correlation excludes the one service per application that has no caller in the graph: nginx-web-server in Social Network and frontend in Hotel Reservation. A gateway receives traffic only from the load generator, which is not a node of the graph, so its ancestor count is zero by construction. Killing it also removes the only entry to the application and leaves the outcome undefined. Without the gateways, 11 services remain in Social Network and 7 in Hotel Reservation.

D. Predictors

Section III defines the caller count and the hybrid metric. The ten further comparators are degree, in-degree, out-degree, betweenness, closeness, eigenvector centrality [30], PageRank [31], a composite rank, descendant count and dominator subtree size, the number of services reachable from the gateway only through a given service. Dominators are a classical notion for flow graphs [32]; here they are computed with the immediate-dominator routine of NetworkX. Centralities follow the standard definitions [7], and betweenness uses the algorithm of Brandes [11]. The plan forbade dropping any comparator after results were seen, and none was dropped.

The hybrid metric's weight values, 0.4, 0.4 and 0.2, come from the project's original proposal, where an empirical search had chosen them, so they were not fixed independently of all data. The locked plan called for tuning the weights on Social Network and freezing them for Hotel Reservation. That step was never run (Deviation 10), and no weights were re-tuned on campaign data. A sensitivity analysis over every weighting on the preregistered grid, run after the confirmatory results were known, is reported in Section V-J.

Two predictor-application pairs were declared degenerate in advance because the predictor is constant across the analysis set, which leaves Spearman's ρ undefined. Eigenvector centrality is zero for every analysed Social Network service, since the reconstructed graph is acyclic, and every Hotel Reservation service has exactly one direct caller, so in-degree is constant there. Both pairs are left out of the test family, because a degenerate graph property is not a null result and counting it as one would dilute the correction. That leaves 22 testable pairs of the 24 considered.

E. Workload and Run Protocol

Each run starts the load generator, Locust 2.46.3, with 25 users spawned at 10 per second. Each simulated user waits between 0.5 and 1.5 seconds between requests and draws each request from a fixed weighted mix (Table I). On Social Network the weights are 3 for composing a post, 5 for reading a home timeline and 2 for reading a user timeline. On Hotel Reservation they are 6 for a hotel search, 2 for a reservation, 2 for a user lookup and 1 for recommendations.

The load runs for 60 seconds before the fault, and the fault-free baseline percentiles are computed over that same 60-second window. The window therefore opens when the load generator starts and includes its ramp-up (Deviation 13); its length was the same in every run. The fault is then held for 30 seconds, the recovery probe runs, and a 20-second cooldown separates one run from the next.

Faults are injected with Pumba, which the runner starts as a short-lived container that acts on the target container through the Docker socket; every command goes through the Docker 29.8.0 client. Table II lists the command lines, generated from the runner code. A kill fault sends SIGKILL, which ends the process without draining connections or answering requests in flight. A latency fault applies a `tc netem` rule that adds 500 ms of delay, with 100 ms of jitter, to the container's network interface. The kill asks what happens when a dependency vanishes; the delay asks what happens when it stays up but answers late.

The runner finds the target as the one container whose `com.docker.compose.service` label equals the service name exactly, and it stops if none or several match, so that a fault cannot land on a different service from the one recorded (Deviation 2). Timing differs by fault type. A kill completes at once; the runner waits out the fault window, restores the service with `docker compose up -d`, and takes the fault-removal instant when that command returns. Because `pumba netem` blocks for its declared duration, the runner launches it as a separate process and takes the fault-removal instant when that process exits. After the recovery probe of every latency run, the runner recreates the target container so that no residual netem rule reaches the next run (Deviation 4). Algorithm 1 gives one run in full, with warm-up and baseline

TABLE II
COMMAND LINES ISSUED IN EACH RUN, GENERATED FROM THE RUNNER CODE

Step	Command	Source
Start the load	`python -m locust -f <locustfile> --host <url> --headless --users <users> --spawn-rate 10 --run-time <duration>s --only-summary --loglevel ERROR`	runner, load start
Find the target container	`docker ps --filter label=com.docker.compose.service=<service> --format '{{.Names}}\{}t{{.Label "com.docker.compose.service"}}'`	runner, target resolution
Kill fault	`docker run --rm -v /var/run/docker.sock:/var/run/docker.sock ghcr.io/alexei-led/pumba kill --signal SIGKILL <container>`	fault configuration
Latency fault	`docker run --rm -v /var/run/docker.sock:/var/run/docker.sock ghcr.io/alexei-led/pumba netem --duration 30s delay --time 500 --jitter 100 <container>`	fault configuration
Restore after a kill	`docker compose up -d <service>`	runner, restore
Clean up after a latency fault	`docker compose up -d --force-recreate <service>`	runner, clean-up

Algorithm 1 One campaign run (`one_run` in `tools/run_pilot.py`), called in manifest order with a cooldown after each run

Input: application a, service s, fault type f, repetition r, graph G of a

$c \leftarrow$ the running container whose label `com.docker.compose.service` equals the compose name of s; stop unless exactly one matches
start the Locust load against the gateway of a
wait W; $B \leftarrow$ per-service baseline percentiles and error rates over the last W
$t_{\text{inj}} \leftarrow$ now; issue the Pumba command for f on c ▷ kill: synchronous; latency: background process
wait H; $t_{\text{end}} \leftarrow$ now
$D \leftarrow$ services degraded in $[t_{\text{inj}}, t_{\text{end}}]$ relative to B
persist the raw spans of the baseline and fault windows
if f = kill **then**
    run `docker compose up -d` for s
else
    wait for the Pumba process to exit
end if
$t_{\text{rm}} \leftarrow$ now ▷ fault-removal instant
$T_{\text{rec}} \leftarrow$ recovery probe from t_{rm} with interval P, confirmation K, lag L and cap C
if $f \neq$ kill **then**
    run `docker compose up -d --force-recreate` for s
end if
classify D into ancestors, descendants and unrelated services of s in G
stop the load; **return** the run record

window W of 60 s, fault hold H of 30 s, probe interval P of 1 s, confirmation period K of 5 s, probe lag L of 5 s and recovery cap C of 180 s.

The design is fully crossed and balanced. Each analysable service receives each fault type 5 times, giving 36 cells and 180 runs, 110 on Social Network and 70 on Hotel Reservation. A run with incomplete telemetry is discarded and repeated in its own cell; it is never replaced elsewhere and never analysed as zeros. The same rule covered the 17 runs invalidated by Deviations 1, 2 and 4, which were collected again in their cells. The campaign ran from 2026-09-26 05:02 to 2026-09-26 13:35 UTC, measured from the first to the last end-timestamp of the kept runs, and lost 0 runs to telemetry failure. The raw spans of the baseline and fault windows were stored for every run, so any later change to a metric definition can be replayed on the original data without repeating the experiment.

F. Per-Service Latency and Error Attribution

The latency definition needed more care than any other part of the method, because the obvious implementation is wrong in a way that is easy to miss. A tracing backend asked for the traces involving a service returns every trace in which the service appears, and the root span of each trace times the whole request from end to end. A percentile over those root spans, labelled as the p95 of the service, is really the end-to-end p95 of every request that passed through it, and every service on a request path receives the same value. Error rate has the same flaw if a trace counts as failed whenever any of its spans carries an error. Services on a shared path then become indistinguishable, and a threshold flags them as a group.

Fig. 5 shows the effect on Hotel Reservation under a latency fault in the search service, using 2 pilot validation runs that are not part of the campaign. Under the end-to-end scope, three services, geo, rate, search, share an identical baseline p95 in both runs, and a multiplicative threshold flags five services that were not faulted. Two of them, profile and reservation, are neither ancestors nor descendants of search; they share a trace with it and nothing more.

Each service's latency is therefore measured on the spans that service emits itself. On Hotel Reservation, whose spans carry a span-kind tag, client spans are excluded, because a client span recorded by a service times its callee and would pass a downstream delay back up. Social Network's spans carry no span-kind tag, so all of a service's spans were used there, and 49.7% of the spans used were client spans. A post hoc replay restricted to server spans changed the outcome of 2 of 110 runs and none of the service-level medians, and the primary result came out identical (Deviation 11). An error counts against a service only when one of its own spans is tagged as an error, divided by the number of traces in which the service appears. With per-service attribution, the same two

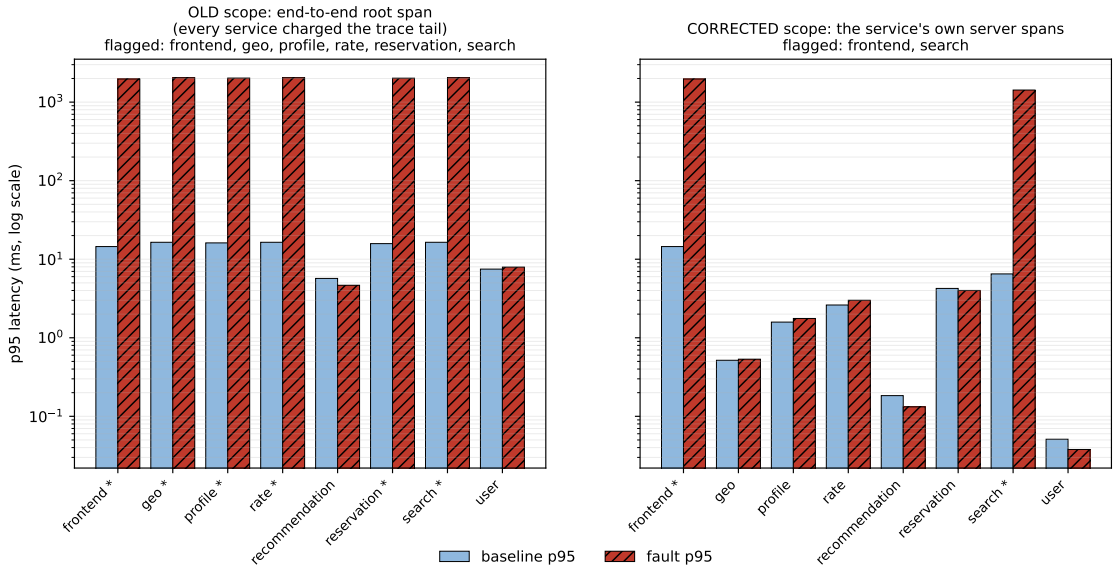


Fig. 5. Per-service p95 on Hotel Reservation under a latency fault in search, measured on the end-to-end root span of each trace (left) and on each service's own spans (right). An asterisk marks a service flagged as degraded.

pilot runs flag one non-faulted service instead of five, and that service is an ancestor of search. The services that drop out keep a flat service time while the end-to-end value charged to them grows more than a hundredfold, the signature of a measurement artifact; shared-resource contention would raise their own service times.

G. Outcomes: Blast Radius and Recovery Time

A service counts as degraded when either of two conditions holds on its own spans during the fault window. The latency condition requires its p95 to exceed 2.0× its baseline p95 and to rise by at least 1.0 ms. The error condition requires its error rate to rise by at least 0.05 in absolute terms. The absolute floor became necessary once percentiles were truly per service: some services here have baseline p95 values in the hundredths of a millisecond, where a purely multiplicative rule reacts to scheduler noise (Deviation 1). Both conditions are evaluated at full precision. Degraded services are split into the ancestor, descendant and unrelated counts of Section III.

Which indicator carries the signal depends on the fault type, and the plan fixed the pairing in advance. Under kill faults the primary indicator is error rate, because requests fail; in the pilot the sign of the kill-fault change in p95 even differed between the two applications, so it is never pooled. Under latency faults the change in p95 is the primary indicator and error rate carries no signal, because nothing has failed.

Recovery time, T_{rec}, is the number of seconds from fault removal until the changes in p95, p99, error rate and downstream-affected count are all back within 10% of their baseline and stay there for a 5-second confirmation period. Timing from removal matters, since timing from injection would include the whole fault and could never fall below it. The probe polls at a nominal 1-second interval, with an effective resolution of 1.4 to 2.4 seconds measured as the median gap between samples within each run. Each sample reads a window lagged 5 seconds into the past, because tracing backends flush spans in batches, and needs at least 20 traces before its percentiles can vote for recovery. A run that has not recovered within 180 seconds is recorded as right-censored and never filled in with a number. No run was censored. Observed recovery times ranged from 9.72 s to 156.27 s, with a median of 15.74 s. The plan stated in advance that a low censoring rate would make the rank test the primary test for T_{rec}, with survival models reported beside it; at a censoring rate of 0.0%, that rule applied (Deviation 6).

H. Statistical Analysis

Associations between a graph predictor and a measured outcome are tested with Spearman's ρ at the service level, after taking the median over each service's repetitions; the repetitions make each service's value more precise but add no points to the correlation. Significance comes from permutation tests, exact where all orderings can be enumerated and otherwise from 10,000 random permutations, with one added to both counts so that no p-value is zero [33]. The asymptotic approximation is unreliable at these sample sizes and with the ties these graphs produce. Confidence intervals are bias-corrected and accelerated bootstrap intervals [34], with a percentile interval, marked as such, where ties leave the bias correction undefined. For recovery time the Cox proportional hazards model [35] and the log-rank test serve as secondary analyses. The analysis ran on Python 3.13.2, with SciPy 1.14.1 supplying ranks and the reference Spearman implementation and statsmodels 0.14.6 the survival models.

Tests fall into three families, each corrected separately with the Benjamini-Hochberg procedure at $q = 0.05$ [29]. The confirmatory family holds 44 tests on kill faults: the 22 testable pairs, each against Y and against T_{rec}. The replication family repeats them on latency faults. A third, exploratory family covers secondary outcomes and carries no confirmatory weight. The

TABLE III
SOCIAL NETWORK: GRAPH PREDICTORS AND MEASURED SERVICE MEDIANS FOR EACH ANALYSED SERVICE

Service	Ancestors	Descendants	In-degree	Out-degree	HC	Betweenness	Closeness	PageRank	Eigenvector	Degraded ancestors (kill)	Degraded ancestors (latency)	T_{rec} kill (s)	T_{rec} latency (s)
post-storage-service	4	0	3	0	0.109	0.000	0.291	0.157	0.000	4	4	15.3	15.4
user-service	4	0	2	0	0.073	0.000	0.242	0.145	0.000	2	2	14.5	14.8
social-graph-service	3	1	2	1	0.081	0.014	0.205	0.098	0.000	3	3	15.0	15.4
url-shorten-service	3	0	1	0	0.036	0.000	0.136	0.080	0.000	3	3	16.1	15.2
user-mention-service	3	0	1	0	0.036	0.000	0.136	0.080	0.000	3	3	14.5	14.4
home-timeline-service	2	3	2	2	0.096	0.012	0.182	0.074	0.000	2	2	15.4	15.6
media-service	2	0	1	0	0.036	0.000	0.121	0.061	0.000	2	2	16.1	14.8
text-service	2	2	1	2	0.064	0.036	0.121	0.061	0.000	2	2	15.3	14.4
unique-id-service	2	0	1	0	0.036	0.000	0.121	0.061	0.000	2	2	14.2	14.6
user-timeline-service	2	1	2	1	0.079	0.003	0.182	0.066	0.000	2	2	15.7	14.9
compose-post-service	1	10	1	7	0.302	0.053	0.091	0.062	0.000	1	1	15.6	15.6

plan considered pooling the families into one correction and rejected it, because the confirmatory and replication families ask the same question under different faults. Two gates run before any statistic: one confirms that every expected service was present in every run, and the other compares each run's fault-free baseline with those of all other runs, using the median and the median absolute deviation. All 10 pre-analysis integrity checks passed.

I. Preregistration and Deviations

The plan was locked before the campaign started [9], and every later change was logged with its date and with whether campaign data had been analysed by then. 13 deviations were recorded; Appendix A lists them. 10 invalidated no data. The other 3 invalidated 17 runs, each collected again in its own cell, so the final dataset is complete and balanced. Deviation 1 added the absolute latency floor after the pre-launch rehearsal, Deviation 2 replaced a substring match that had faulted the wrong container, and Deviation 4 dealt with a netem rule that outlived its run and inflated later baselines. The remaining deviations concern the analysis or its reporting: the rank test becoming primary for recovery time, an asymptotic p-value quoted in the plan, exact p-values reported only as supplementary, the weight tuning that was not run, the Social Network span selection, and Deviation 13, found while this paper was written, which records that the baseline window includes the load ramp-up; it invalidated no run, and the baseline gate found no anomalies.

V. RESULTS

A. Campaign Completion

All 180 planned runs completed across 36 balanced cells. No run was lost to telemetry failure, no gateway was faulted, and every recovery time was observed. Tables III and IV list the graph predictors and the measured outcomes of every analysed service.

B. The Confirmatory Test of H1

On Social Network under kill faults, the caller count X and the number of degraded callers Y correlate at $\rho = 0.785$ across 11 services, with a bias-corrected and accelerated bootstrap interval from 0.000 to 1.000. The uncorrected p-value from 10,000 random permutations is 0.0080. After Benjamini-Hochberg correction across the 44 tests of the confirmatory family, the adjusted p-value is 0.3451. Exact enumeration, run after the results were known (Deviation 9), gives 0.006926 uncorrected and 0.3048 adjusted, and the verdict does not change.[1]

Not one of the 44 confirmatory tests reaches significance at $q = 0.05$. The primary test has the smallest uncorrected p-value in the family, and its adjusted value is about 6.9 times the threshold. Table V gives the primary test for each family and application, and Table VI summarises both families. H1 is not confirmed. The point estimate is large and positive, the correction fixed in advance rejects it, and the interval runs from no association to a perfect one.

Two features of the data qualify the estimate further, and they matter more than its p-value. The first is saturation. Since Y counts degraded callers and X counts callers, Y can never exceed X, and if every service's measured count equalled its graph count the correlation would be one by construction and would test nothing. On Hotel Reservation that is what happened: each service's measured outcome matched its predictor in every run, under both fault types. Hotel Reservation is therefore tautological at the unit of analysis, and every Hotel Reservation result is reported as directional only.

Social Network is not fully saturated. Its run-level saturation under kill faults is 0.909, which is why ρ falls short of one. Fig. 6 shows the pattern: ten of the eleven services lie exactly on the line $y = x$. One departs from it. User-service has 4 graph ancestors and registered 2 degraded ancestors in all 5 of its kill repetitions. The empirical content of the Social Network correlation, as distinct from its definitional content, rests on that one service. A leave-one-out recomputation, designed after the confirmatory result was known and carrying no inferential weight, withholds one service at a time (Fig. 7). The correlation then ranges from 0.696 to 1.000, and the value of 1.000 comes from withholding user-service.

C. The Hybrid Metric and H2

The hybrid metric combines inbound centrality with a fan-out-scaled outbound term, on the premise that a service calling many others matters because its failure breaks many

[1]The locked Monte Carlo values remain the preregistered result.

TABLE IV
HOTEL RESERVATION: GRAPH PREDICTORS AND MEASURED SERVICE MEDIANS FOR EACH ANALYSED SERVICE

Service	Ancestors	Descendants	In-degree	Out-degree	HC	Betweenness	Closeness	PageRank	Eigenvector	Degraded ancestors (kill)	Degraded ancestors (latency)	T_{rec} kill (s)	T_{rec} latency (s)
geo	2	0	1	0	0.057	0.000	0.190	0.152	0.707	2	2	22.0	17.7
rate	2	0	1	0	0.057	0.000	0.190	0.152	0.707	2	2	20.9	16.9
profile	1	0	1	0	0.057	0.000	0.143	0.124	0.002	1	1	27.0	18.7
recommendation	1	0	1	0	0.057	0.000	0.143	0.107	0.002	1	1	22.6	25.4
reservation	1	0	1	0	0.057	0.000	0.143	0.127	0.002	1	1	19.3	16.6
search	1	2	1	2	0.112	0.048	0.143	0.119	0.002	1	1	22.3	24.7
user	1	0	1	0	0.057	0.000	0.143	0.119	0.002	1	1	19.7	13.6

TABLE V
TEST OF H1, CALLER COUNT X AGAINST DEGRADED CALLERS Y, IN EACH FAMILY AND APPLICATION

Family	Fault	Application	Services	ρ	95% CI	CI method	p (raw)	p method	p (BH)	Significant	Note
confirmatory	kill	Social Network	11	0.785	0.000 to 1.000	BCa	0.0080	Monte Carlo, 10,000 permutations	0.3451	no	
confirmatory	kill	Hotel Reservation	7	1.000	1.000 to 1.000	percentile	0.0476	exact, 5,040 permutations	0.3451	no	saturated; directional only
replication	latency	Social Network	11	0.785	0.000 to 1.000	BCa	0.0076	Monte Carlo, 10,000 permutations	0.1848	no	
replication	latency	Hotel Reservation	7	1.000	1.000 to 1.000	percentile	0.0476	exact, 5,040 permutations	0.2328	no	saturated; directional only

TABLE VI
FAMILY SIZES AND THE NUMBER OF TESTS THAT SURVIVE BENJAMINI-HOCHBERG CORRECTION

Family	Fault	Tests	Tests with defined p	Significant after BH	Smallest p (raw)	Its p (BH)	Test with the smallest p
confirmatory	kill	44	44	0	0.0080	0.3451	Social Network, caller count X against Y
replication	latency	44	44	0	0.0076	0.1848	Social Network, caller count X against Y

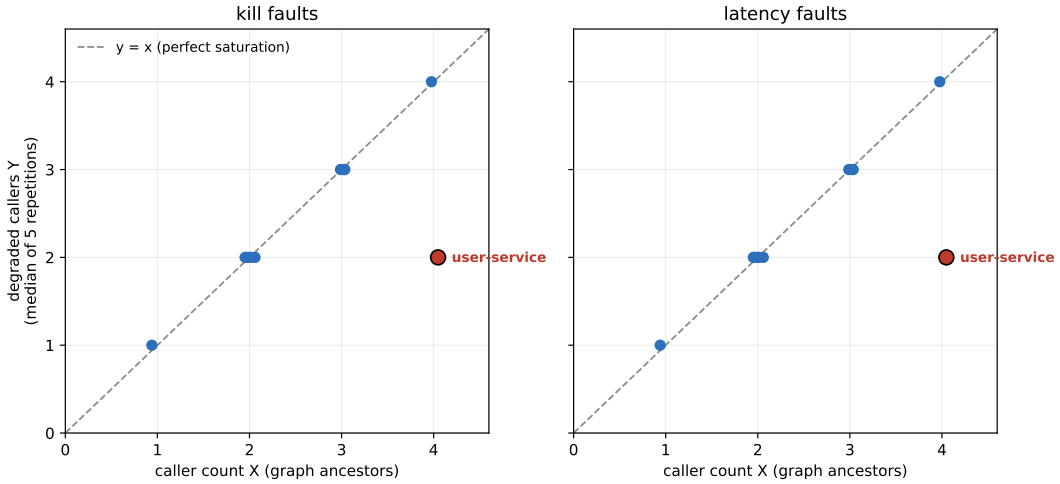


Fig. 6. Caller count X against the measured number of degraded callers Y for each Social Network service under kill and latency faults. Points on the dashed line $y = x$ are saturated; the labelled service departs from it.

downstream links. H2 expected that premise to point the wrong way. Table VII gives all 4 application and fault combinations. They are not independent: the outcome vectors coincide across fault types, so the four cells hold 2 distinct comparisons, one per application.

The correlation between the hybrid metric and Y is negative in every cell, $\rho = -0.144$ on Social Network under both kill and latency faults and $\rho = -0.258$ on Hotel Reservation. After correction no cell can be told apart from zero. The adjusted p-values are 0.9925 and 0.9152 on Social Network and 1.0000 on Hotel Reservation, and the Social Network interval runs from -0.810 to 0.698. The direction H2 states appears every time, but its size cannot be separated from zero, so no negative relationship is claimed. What the data show is an absence of support for the fan-out premise.

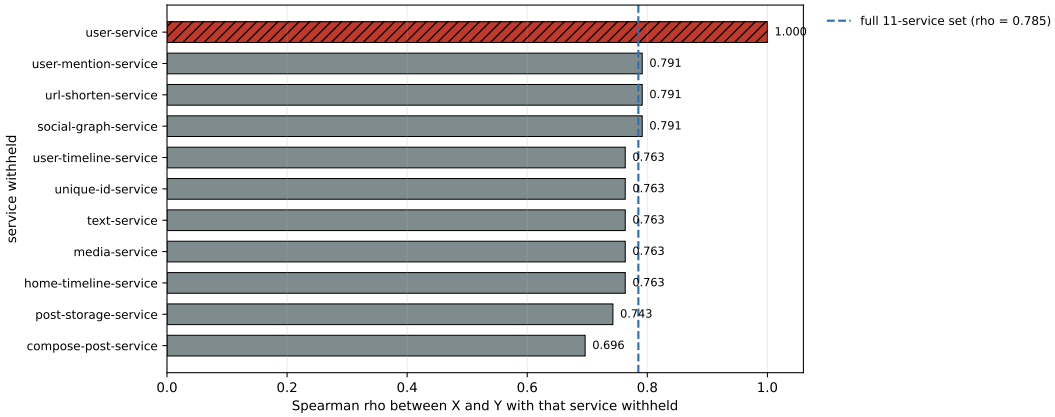


Fig. 7. Post hoc leave-one-out check: Spearman's ρ between X and Y on Social Network under kill faults, recomputed with each service withheld.

TABLE VII
TEST OF H2, THE HYBRID METRIC HC AGAINST DEGRADED CALLERS Y, IN ALL FOUR APPLICATION AND FAULT CELLS

Family	Fault	Application	Services	ρ	95% CI	p (raw)	p (BH)	$\rho \leq 0$	Distinguishable from zero after BH
confirmatory	kill	Social Network	11	−0.144	−0.810 to 0.698	0.6688	0.9925	yes	no
confirmatory	kill	Hotel Reservation	7	−0.258	−0.730 to −0.167	1.0000	1.0000	yes	no
replication	latency	Social Network	11	−0.144	−0.805 to 0.713	0.6656	0.9152	yes	no
replication	latency	Hotel Reservation	7	−0.258	−0.750 to −0.167	1.0000	1.0000	yes	no

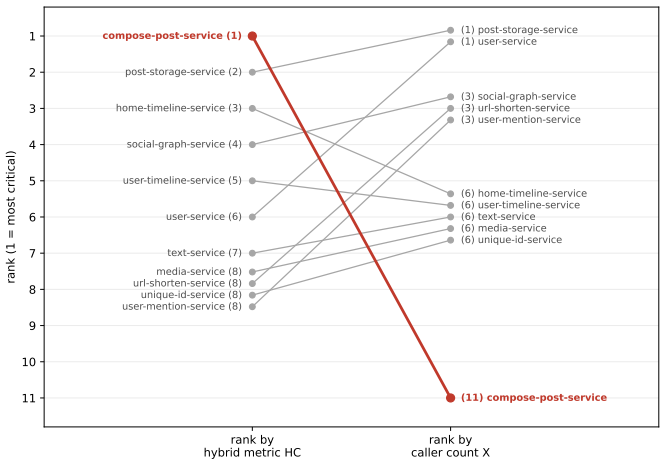


Fig. 8. Rank of the eleven Social Network services by the hybrid metric HC and by caller count X, with compose-post-service highlighted.

D. The Motivating Case

The metric was first motivated by compose-post-service, which calls 7 services directly and has 10 transitive descendants. On the strength of that fan-out, the hybrid metric ranks it first of the 11 analysed services, with a score of 0.302. By transitive callers it ranks last, having a single graph ancestor. Fig. 8 shows the full rank inversion.

Measurement agrees with the caller-based ranking. Faulting compose-post-service left exactly one degraded ancestor in each of its 5 kill repetitions, and the latency runs gave the same result. A post hoc breakdown shows that in all 10 runs that ancestor was nginx-web-server, the gateway. On the pre-registered outcome, the service the original metric rates most critical is the lowest in its application. The outcome counts degraded services, though, and does not measure requests or functionality lost, and this is a single worked case: the test across all eleven services, which could have generalised it, is the null result of Section V-B.

E. Replication under a Second Fault Type

The replication family repeats all 44 tests on latency faults. Again no test is significant after correction. The smallest uncorrected p-value is 0.0076, with an adjusted value of 0.1848. The agreement carries a caveat. Under kill and latency faults the service medians of Y are identical in both applications, and at run level 17 of the 18 services show the same five per-run counts under both fault types. On the primary outcome the latency family therefore repeats the kill family's correlations on the same values and adds no independent evidence about H1.

Read on its own, this says that which services degrade when a service fails hardly depends on how it fails: a process that disappears and a process that slows down reach the same neighbours. Recovery differs. The correlation between median recovery times under kill and under latency faults is 0.384 on Social Network and 0.857 on Hotel Reservation, so for recovery time the second fault type does add information.

F. Direction of Propagation

Across the campaign, degradation travelled to callers and not to callees. No descendant of the faulted service degraded in 179 of 180 runs, and at the unit of analysis, the median over a service's repetitions, the count is zero for every service

TABLE VIII
PREREGISTERED POWER ANALYSIS REPRODUCED ON THE REALISED ANALYSIS SETS; PERMUTATION P-VALUES FROM 10000 RANDOM PERMUTATIONS (SOCIAL NETWORK) AND EXACT ENUMERATION OF 5040 PERMUTATIONS (HOTEL RESERVATION)

Quantity	Social Network	Hotel Reservation
Services, realised / preregistered	11 / 11	7 / 7
Distinct X values, realised / preregistered	4 / 4	2 / 2
X values	1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4	1, 1, 1, 1, 1, 2, 2
Ceiling ρ, outcome without ties, realised / preregistered	0.941 / 0.941	0.791 / 0.791
Ceiling p, outcome without ties, permutation	0.0002	0.0952
Ceiling p, outcome without ties, asymptotic (not for inference)	<0.0001	0.0343
Ceiling ρ, outcome tied like X	1.000	1.000
Ceiling p, outcome tied like X, permutation	0.0001	0.0476
p quoted in the preregistration		0.034

under both fault types. The single exception, a kill repetition of compose-post-service on Social Network, recorded 2 degraded descendants. Services unrelated to the faulted one behave the same way, with 179 of 180 runs at zero and one exception.

This is the firmest structural result of the study, and it cannot be written as a correlation. Neither outcome varies between services, so 88 of the 154 exploratory tests attempted have no defined value. A constant outcome gives no correlation at all, which is different from a weak one. These tests are reported as undefined, with the reason, in the same way as the 2 degenerate predictors, and are not entered as zeros.

G. Exploratory Findings

Of the 66 testable exploratory tests, 2 survive correction within that family. Both concern the size of latency inflation under latency faults on Social Network: the change in p95 correlates with the hybrid metric at $\rho = 0.949$ and with plain degree at $\rho = 0.953$, both with an adjusted p of 0.0033. These results carry no confirmatory weight, and their outcome is not the one H2 concerns. How much latency inflates differs from how many callers degrade, and plain degree does at least as well as the hybrid metric on it, which gives no reason to prefer the fan-out correction.

H. What the Design Could Have Detected

Table VIII reproduces the preregistered power analysis on the realised analysis sets. Both sets have the registered size and the registered number of distinct predictor values, and the ceiling correlations come out exactly as preregistered. The caller count takes only 4 distinct values across the 11 Social Network services and 2 across the 7 Hotel Reservation services. For an outcome without ties, these ties cap the attainable correlation at 0.941 and 0.791.

For Hotel Reservation the limit is severe. An outcome without ties can reach at best $\rho = 0.791$, with an exact permutation p of 0.0952, above the conventional threshold before any correction. The preregistration quoted 0.034 for this ceiling, which is the asymptotic value; the exact value the plan requires is the one above (Deviation 7). Only an outcome that repeats the predictor's own ties can do better, reaching $\rho = 1.000$ with p = 0.0476, and saturation produced exactly that outcome (Table V). Hotel Reservation could therefore pass the nominal threshold before correction only by restating its predictor, and after correction its primary test is far from significant. That follows from its tie structure and its size, not from anything measured.

I. Recovery Time

No run was censored, so under the preregistered rule the Spearman rank test is the primary test for recovery time, and the Cox and log-rank models are secondary and uncorrected (Deviation 6). Table IX gives all three for both predictors. No recovery-time test is significant after correction in either family. The log-rank test is undefined for 10 of the 22 recovery-time tests in each family, among them X on Hotel Reservation, where the predictor has fewer than two distinct terciles. Supplementary file S2 gives the unrounded estimates.

J. Post Hoc Weight Sensitivity

The confirmatory results were already in hand when this analysis was designed, and it belongs to no preregistered family. It asks how much the hybrid metric's association with the outcome depends on its weights. The metric was recomputed for all 231 weight triplets on the preregistered grid, with step 0.05 and weights summing to one, using the same code, the same service medians and the same permutation tests. Each set of 231 tests was corrected with the Benjamini-Hochberg procedure within that set. No triplet was selected or adopted, and every other result in this paper uses the reported weights. Table X and Fig. 9 give the full distribution.

Against the number of degraded callers on Social Network, ρ ranged from -0.544 to 0.337, with a median of -0.167, and was positive for 28 of the 231 triplets. No triplet reached an uncorrected p below 0.05; the smallest was 0.0952, and none survived correction. Kill and latency faults gave identical results, since the service medians coincide. On Hotel Reservation every triplet with a defined correlation, 230 in all, gave $\rho = -0.258$, because a weighting on in-degree alone leaves the metric constant there. The reported weights gave $\rho = -0.144$ on Social Network, and 47 triplets gave a larger value. The null result for H2 therefore does not hinge on the inherited weights: no weighting on the grid produced a significant association with degraded callers in either direction.

Against the gateway's error rate under kill faults, an exploratory outcome, the Social Network picture differs. ρ was positive for all 231 triplets, from 0.229 to 0.832 with a median of 0.753. Of these, 145 had uncorrected p-values under 0.05, and 128 remained significant after correction within the set; the p-values come from 10,000 random permutations, because this outcome has too many distinct orderings to enumerate. At the reported weights ρ was 0.786. The largest value is the best of 231 tries on one dataset and is not a finding. On Hotel Reservation every defined triplet gave $\rho = -0.204$ and none was significant. At the reported weights, the preregistered

TABLE IX

RECOVERY TIME: SPEARMAN'S ρ (PRIMARY, BH-CORRECTED WITHIN EACH FAMILY) AND COX AND LOG-RANK TESTS (SECONDARY, UNCORRECTED). HAZARD RATIOS ARE PER ONE-UNIT INCREASE OF THE PREDICTOR, FITTED PER RUN WITH SERVICE-CLUSTERED ERRORS AND PER SERVICE; THE HYBRID METRIC SPANS LESS THAN ONE UNIT, SO ITS HAZARD RATIOS EXTRAPOLATE BEYOND THE OBSERVED RANGE. LOG-RANK IS UNDEFINED WHERE THE PREDICTOR HAS FEWER THAN TWO DISTINCT TERCILES

Fault	Application	Predictor	ρ	p (raw)	p (BH)	HR per unit, run	95% CI, run	p, run	HR per unit, service	95% CI, service	p, service	Log-rank p
kill	Social Network	caller count X	−0.356	0.2807	0.9264	1.173	0.876 to 1.570	0.2836	1.479	0.688 to 3.181	0.3163	0.5154
kill	Social Network	hybrid metric HC	0.098	0.7699	0.9925	0.010	0.002 to 0.043	<0.0001	1.408	5.49×10^{-4} to 3.61×10^{3}	0.9319	0.2099
kill	Hotel Reservation	caller count X	−0.158	0.8571	0.9925	1.522	1.067 to 2.171	0.0204	2.179	0.302 to 15.722	0.4399	undefined
kill	Hotel Reservation	hybrid metric HC	0.204	0.8571	0.9925	0.147	8.91×10^{-4} to 24.134	0.4609	not estimable	not estimable	not estimable	undefined
latency	Social Network	caller count X	−0.060	0.8647	1.0000	0.998	0.816 to 1.221	0.9859	1.184	0.627 to 2.236	0.6018	0.5561
latency	Social Network	hybrid metric HC	0.767	0.0084	0.1848	0.045	0.002 to 1.163	0.0616	2.49×10^{-4}	8.19×10^{-10} to 75.493	0.1975	0.1099
latency	Hotel Reservation	caller count X	−0.158	0.8571	1.0000	0.882	0.578 to 1.346	0.5609	2.179	0.302 to 15.722	0.4399	undefined
latency	Hotel Reservation	hybrid metric HC	0.408	0.5714	0.8111	6.26×10^{-4}	1.07×10^{-6} to 0.368	0.0234	not estimable	not estimable	not estimable	undefined

Not estimable: the Cox fit did not converge or showed separation because the predictor takes few distinct values; raw estimates are in supplementary file S2.

TABLE X

POST HOC WEIGHT SENSITIVITY OF THE HYBRID METRIC OVER 231 WEIGHT TRIPLETS, WITH BH CORRECTION WITHIN EACH SET OF 231

Application	Fault	Outcome	Triplets with defined ρ	ρ min	ρ median	ρ max	$\rho > 0$	p (raw) < 0.05	Significant after BH	ρ at 0.4/0.4/0.2	p at 0.4/0.4/0.2
Social Network	kill	degraded callers Y	231	−0.544	−0.167	0.337	28	0	0	−0.144	0.6695
Social Network	latency	degraded callers Y	231	−0.544	−0.167	0.337	28	0	0	−0.144	0.6695
Hotel Reservation	kill	degraded callers Y	230	−0.258	−0.258	−0.258	0	0	0	−0.258	1.0000
Hotel Reservation	latency	degraded callers Y	230	−0.258	−0.258	−0.258	0	0	0	−0.258	1.0000
Social Network	kill	gateway error rate	231	0.229	0.753	0.832	231	145	128	0.786	0.0064
Hotel Reservation	kill	gateway error rate	230	−0.204	−0.204	−0.204	0	0	0	−0.204	0.8571

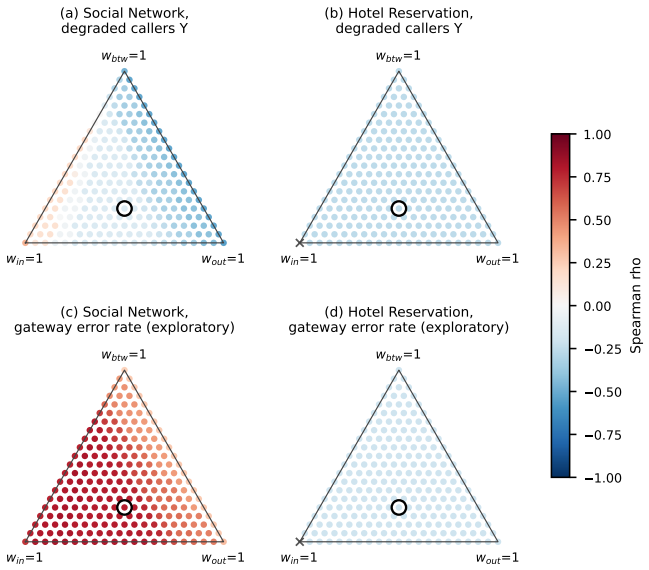


Fig. 9. Post hoc weight sensitivity: Spearman's ρ of the hybrid metric against degraded callers (a, b) and the gateway error rate (c, d) for every weight triplet under kill faults; latency faults give identical results for degraded callers. Circled: the reported weights; cross: metric constant.

exploratory test of this association did not survive correction across its exploratory family (adjusted $p = 0.0913$), and Section VI-C shows that the association follows the metric's in-degree term.

K. Verification

Two checks beyond the preregistered analysis are reported as supplementary. One is the post hoc leave-one-out recomputation of Section V-B. The other is an independent reimplementation of the primary statistic, written without reference to the analysis code and reading only the stored spans and the dependency graph. It reproduces the reported correlation, and it agrees with the stored per-run outcomes on which services were affected, not only how many, in all 110 Social Network runs. That shows the analysis code is faithful to its data. Whether the definitions of Sections IV-F and IV-G are the right ones is a question no recomputation can settle.

VI. DISCUSSION

A. Why Fan-Out Points the Wrong Way

The direction of propagation is the result that needs the fewest qualifications. In 179 of 180 runs no descendant of the faulted service degraded, and which services degraded hardly depended on how the fault was delivered: for 17 of the 18 services the per-run counts were identical whether the service was killed or slowed down. The most plausible reading is that a caller suffers while it waits on a dependency that has died or slowed, whereas a callee only receives fewer requests and has nothing to wait for. The measurements fit that reading without testing it, because the degradation rule asks only whether a service's own latency or error rate crossed a threshold and never observes waiting directly.

If the reading is right, a fan-out ranking scores the wrong end of each edge. The composition service of Social Network is the clearest case: it calls 7 services and the fan-out metric ranks it first, yet in each of its 10 runs exactly one caller degraded, and that caller was always the gateway. Across all services, the metric's correlation with degraded callers was negative in every cell and separable from zero in none (Section V-C), and no weighting on the preregistered grid

changed that (Section V-J). The data give the fan-out premise no support; they cannot show a negative relationship.

B. A Predictor That Sits Close to Its Outcome

The caller count correlates with degraded callers partly by construction, since the outcome counts a subset of the predictor, and the two are equal whenever every ancestor of a faulted service degrades. That held for every Hotel Reservation service and for ten of the eleven Social Network services. A post hoc breakdown sharpens the point. In all 180 runs the gateway was among the degraded ancestors, and in both applications every analysable service has the gateway among its graph ancestors. Every measured ancestor count therefore includes one unit that is the same for every service, and what remains to be explained comes from intermediate callers. In Social Network only user-service departs from its graph count in a consistent way. Its two ancestors that degraded sit on request paths the workload exercises, and its other two showed no degradation in any run, an explanation formed after seeing the data and not tested here.

With user-service withheld, the Social Network correlation becomes 1.000. The result can fairly be read as resting on one service's worth of independent information. One lesson may reach beyond this study: a structural predictor tested against a structurally defined outcome needs a saturation measure reported next to the correlation, without which the correlation cannot be interpreted.

C. What Users Would See

The preregistered outcome counts degraded services and says nothing about how many requests fail. To come closer to what a user experiences, the gateway's end-to-end error rate under kill faults was examined after the fact. This analysis is exploratory and descriptive.

Gateway error followed the request mix. For each faulted service, the Locust task weights of the request types whose traces pass through it were summed and divided by the total weight. The rank correlation between that share and the median gateway error was 0.788 on Social Network and 0.964 on Hotel Reservation, reported without p-values. In Hotel Reservation, killing reservation, which sits on request types carrying 0.727 of the weight, gave a median gateway error rate of 0.7340, while killing recommendation, with a share of 0.091, gave 0.0860. The mix does not explain everything. The Social Network services reached only through the compose request all share a weight of 0.300, and their median error rates still range from 0.1692 to 0.2571.

The hybrid metric's exploratory correlation with gateway error on Social Network was 0.786, not significant after correction within its family (adjusted $p = 0.0913$). Splitting the metric into its weighted terms shows where the association comes from: the in-degree term alone has a rank correlation of 0.715 with gateway error, the fan-out term 0.310 and the betweenness term 0.229. The association follows the count of direct callers more closely than the fan-out correction, so it lends that correction no support.

D. Lessons about Measurement

Two measurement choices mattered more than the plan anticipated. The first is where latency is attributed. Charging each trace's end-to-end latency to every service on the trace makes services on a shared path indistinguishable; in the pilot comparison of Section IV-F that shortcut flagged five non-faulted services where per-service attribution flagged one. The correction was in place before the campaign, so what an uncorrected campaign would have concluded is unknown, but the pilot shows how it would have behaved.

The second is that per-service attribution is only as good as the instrumentation behind it. Social Network's spans carry no span-kind tag, so its per-service latency drew on all of a service's spans, 49.7% of them client spans. A post hoc replay on server spans altered 2 of the 110 run outcomes and none of the service-level medians. The change was small, which was fortunate. A study that depends on span semantics should check them before collecting data, and this one did not.

E. Implications for Practitioners

For a team that has to rank services before running any experiment, the data favour looking up the graph at callers over looking down it at callees. That advice rests on the propagation pattern and on the lack of support for fan-out; the preregistered test of the caller ranking itself did not reach significance, so no claim is made that it predicts failure impact. In both applications the gateway appeared in every caller count, and in the exploratory analysis the user-visible error rate tracked how much traffic passed through the failed service. A ranking that weights callers by the traffic they carry may therefore be worth testing as a new preregistered hypothesis; it was not tested here. Larger applications would also lift the ceiling that eleven services place on any correlation, and containment mechanisms such as retries with back-off or circuit breakers, which were neither configured nor studied, could change how far a failure travels.

VII. Threats to Validity

A. Sample Size and Ties

The unit of analysis is the service, so the sample sizes are 11 for Social Network and 7 for Hotel Reservation, the number of services the applications have. Both were registered in advance and matched exactly. Repetition makes each service's outcome more precise, which is why each cell was run 5 times, but it adds no points to a correlation. Ties tighten the bound further: the caller count takes 4 distinct values on Social Network and 2 on Hotel Reservation, which caps the correlation for an outcome without ties at 0.941 and 0.791. These ceilings were stated before data collection, so they are a property of the design and not an explanation found afterwards. On Hotel Reservation the outcome was also fully saturated, so every result there restates how the outcome was built and is reported as directional only. Its value is as a second, differently shaped topology on which the direction result also holds.

B. A Predictor Close to Its Outcome

The most serious threat to the primary result is conceptual. The outcome counts degraded transitive callers and the predictor counts transitive callers, so they coincide whenever every ancestor degrades, and on Social Network the whole gap between $\rho = 0.785$ and a correlation of one rests on user-service. Withholding it raises ρ to 1.000, a change 2.4 times the size of the next largest, in a check specified after the result was known. The saturation measure was registered before collection precisely to detect this, and the test remained meaningful because saturation was not guaranteed in advance and did not fully hold. A reader may still treat the Social Network correlation as one service's worth of evidence.

C. Benchmarks, Workload and Graph

Both applications come from one benchmark suite at one commit, chosen because they are instrumented and reproducible, not because they sample the space of microservice architectures. The workload is synthetic, fixed at 25 users with a fixed request mix, and containment mechanisms were not studied. Nothing here describes production behaviour. Because the dependency graph is rebuilt from observed traffic, it is a property of deployment and workload together; production call graphs change from request to request [23]. Even here, the traces recover 6 of the 8 edges implied by Hotel Reservation's documentation, and the user service has one observed caller where the documentation implies two. A different request mix would give it a different predictor value with nothing about the deployed system changed, which limits how far any relationship reported here carries to other traffic.

D. Measurement

For latency faults, the runner infers that the netem rule is gone from the exit of the injecting process. That inference failed at least once (Deviation 4). The cross-run baseline gate added afterwards catches a rule that outlives its run, because later baselines then look contaminated, but a rule that leaked and cleared within a single run would pass unnoticed, so latency-fault recovery times carry an assumption the instrumentation cannot check. Blast-radius counts are less exposed, since they come from the fault window itself. The server-span replay of Deviation 11 could not cover recovery times on Social Network, because the probe's windows were not stored, and those times stand as measured. The baseline window includes the load ramp-up (Deviation 13); the same procedure ran in every run and the baseline gate reported no exceptions, but the size of any start-up effect was not measured. Two fault types were studied, a process kill and a 500 ms delay; CPU contention was evaluated in the pilot and dropped because its effect was too small to classify reliably. Finally, some testbed values in Table I were queried after the campaign, on 2026-10-20, and no record from the campaign days confirms that the host operating system build and Docker Desktop version were unchanged.

VIII. Conclusion

A preregistered fault-injection campaign of 180 balanced runs on two benchmark applications asked whether a service's position in its dependency graph indicates how much damage its failure does. The design crossed every analysable service with two fault types and 5 repetitions, and the analysis was locked before the first run.

The correlational hypothesis did not survive. The number of transitive callers correlated strongly with the number of degraded callers on Social Network, but after correction no test survived in either family, the 44 confirmatory and the 44 replication tests alike. The fan-out-weighted metric this project began with correlated negatively in every cell and could not be separated from zero in any, and a post hoc sweep over 231 weightings found none with a significant association with degraded callers.

What held is structural. Failures reached callers and almost never callees: no descendant degraded in 179 of 180 runs, and the service-level median was zero in every condition. That result needs no inference, and it held in both applications and under both fault types.

For practitioners, the evidence points towards ranking services by their callers and away from ranking them by fan-out, with the caveat that the study's own test of the caller ranking did not reach significance. For researchers, it shows two steps that are easy to skip: attributing latency to each service's own spans, and measuring how far an outcome merely restates a structural predictor before reporting a correlation between them. Every number in this paper can be regenerated from the logged campaign data.

Data Availability

The code, the raw campaign data (trace spans for every run, run records and the reconstructed dependency graphs) and the analysis outputs are available at https://github.com/Daxx133-bot/proj-1-updated and will be archived at Zenodo (DOI: [TO FILL]). A single build script regenerates every number, table and figure in this paper from the logged data. Three supplementary files accompany the paper in the repository: S1, the leave-one-out recomputation of Section V-B (`paper/tables/tbl_leave_one_out.csv`); S2, the unrounded recovery-time estimates of Section V-I (`paper/tables/tbl_trec_supplementary.csv`); and S3, the exact-enumeration p-values for the confirmatory family (`analysis/final/exact_p_supplement.csv`). Code is released under the MIT licence, and data and figures under CC BY 4.0.

Acknowledgment

The authors used an AI assistant (Claude, Anthropic) for code development support and language editing. The authors performed all experiments and verified all content.

References

[1] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, Today, and Tomorrow," in *Present and Ulterior Software Engineering (Springer)*, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.

[2] P. Jamshidi, C. Pahl, N. C. Mendonca, J. Lewis, and S. Tilkov, "Microservices: The Journey So Far and Challenges Ahead," *IEEE Software*, vol. 35, no. 3, pp. 24–35, 2018, doi: 10.1109/ms.2018.2141039.

[3] J. Soldani, D. A. Tamburri, and W. J. Van Den Heuvel, "The pains and gains of microservices: A Systematic grey literature review," *Journal of Systems and Software*, vol. 146, pp. 215–232, 2018, doi: 10.1016/j.jss.2018.09.082.

[4] R. Heinrich, A. van Hoorn, H. Knoche, F. Li, L. E. Lwakatare, C. Pahl, S. Schulte, and J. Wettinger, "Performance Engineering for Microservices: Research Challenges and Directions," in *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, 2017, pp. 223–226, doi: 10.1145/3053600.3053653.

[5] H. S. Gunawi, M. Hao, R. O. Suminto, A. Laksono, A. D. Satria, J. Adityatama, and K. J. Eliazar, "Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages," in *Proceedings of the Seventh ACM Symposium on Cloud Computing*, 2016, pp. 1–16, doi: 10.1145/2987550.2987583.

[6] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021, doi: 10.1109/TSE.2018.2887384.

[7] L. C. Freeman, "Centrality in Social Networks: Conceptual Clarification," *Social Networks*, vol. 1, no. 3, pp. 215–239, 1978, doi: 10.1016/0378-8733(78)90021-7.

[8] T. Zimmermann and N. Nagappan, "Predicting Defects Using Network Analysis on Dependency Graphs," in *Proceedings of the 30th International Conference on Software Engineering (ICSE '08)*, 2008, pp. 531–540, doi: 10.1145/1368088.1368161.

[9] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The Preregistration Revolution," in *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, 2018, pp. 2600–2606, doi: 10.1073/pnas.1708274114.

[10] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, 2019, pp. 3–18, doi: 10.1145/3297858.3304013.

[11] U. Brandes, "A Faster Algorithm for Betweenness Centrality," *The Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001, doi: 10.1080/0022250X.2001.9990249.

[12] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy2008), G. Varoquaux, T. Vaught, J. Millman (Eds.), Pasadena, CA, USA*, 2008, pp. 11–15. [Online]. Available: http://conference.scipy.org/proceedings/SciPy2008/paper_2/

[13] R. Albert, H. Jeong, and A. L. Barabási, "Error and attack tolerance of complex networks," *Nature*, vol. 406, no. 6794, pp. 378–382, 2000, doi: 10.1038/35019019.

[14] A. E. Motter and Y. C. Lai, "Cascade-based attacks on complex networks," *Physical Review E*, vol. 66, no. 6, 2002, doi: 10.1103/physreve.66.065102.

[15] S. P. Ma, C. Y. Fan, Y. Chuang, W. T. Lee, S. J. Lee, and N. L. Hsueh, "Using Service Dependency Graph to Analyze and Test Microservices," in *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*, 2018, pp. 81–86, doi: 10.1109/compsac.2018.10207.

[16] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.

[17] A. Basiri, L. Hochstein, N. Jones, and H. Tucker, "Automating Chaos Experiments in Production," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 31–40, doi: 10.1109/icse-seip.2019.00012.

[18] V. Heorhiadi, S. Rajagopalan, H. Jamjoom, M. K. Reiter, and V. Sekar, "Gremlin: Systematic Resilience Testing of Microservices," in *2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS)*, 2016, pp. 57–66, doi: 10.1109/ICDCS.2016.11.

[19] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, and R. Padhye, "Service-Level Fault Injection Testing," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 388–402, doi: 10.1145/3472883.3487005.

[20] P. Alvaro, J. Rosen, and J. M. Hellerstein, "Lineage-driven Fault Injection," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 331–346, doi: 10.1145/2723372.2723711.

[21] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao, "Gray Failure: The Achilles' Heel of Cloud-Scale Systems," in *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, 2017, pp. 150–155, doi: 10.1145/3102980.3103005.

[22] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google Technical Report dapper-2010-1, Tech. Rep., 2010. [Online]. Available: https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/

[23] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, and C. Xu, "Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, 2021, pp. 412–426, doi: 10.1145/3472883.3487003.

[24] M. Chen, E. Kiciman, E. Fratkin, A. Fox, and E. Brewer, "Pinpoint: problem determination in large, dynamic Internet services," in *Proceedings International Conference on Dependable Systems and Networks*, 2002, pp. 595–604, doi: 10.1109/dsn.2002.1029005.

[25] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint Performance Issues with Causal Graphs in Micro-service Environments," in *Lecture Notes in Computer Science*, 2018, pp. 3–20, doi: 10.1007/978-3-030-03596-9_1.

[26] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root Cause Localization of Performance Issues in Microservices," in *NOMS 2020 - IEEE/IFIP Network Operations and Management Symposium*, 2020, pp. 1–9, doi: 10.1109/NOMS47738.2020.9110353.

[27] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, and C. Delimitrou, "Seer: Leveraging Big Data to Navigate the Complexity of Performance Debugging in Cloud Microservices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 19–33, doi: 10.1145/3297858.3304004.

[28] J. Soldani and A. Brogi, "Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022, doi: 10.1145/3501297.

[29] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995, doi: 10.1111/j.2517-6161.1995.tb02031.x.

[30] P. Bonacich, "Power and Centrality: A Family of Measures," *American Journal of Sociology*, vol. 92, no. 5, pp. 1170–1182, 1987, doi: 10.1086/228631.

[31] S. Brin and L. Page, "The anatomy of a large-scale hypertextual Web search engine," *Computer Networks and ISDN Systems*, vol. 30, no. 1-7, pp. 107–117, 1998, doi: 10.1016/s0169-7552(98)00110-x.

[32] T. Lengauer and R. E. Tarjan, "A fast algorithm for finding dominators in a flowgraph," *ACM Transactions on Programming Languages and Systems*, vol. 1, no. 1, pp. 121–141, 1979, doi: 10.1145/357062.357071.

[33] B. Phipson and G. K. Smyth, "Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn," *Statistical Applications in Genetics and Molecular Biology*, vol. 9, no. 1, 2010, doi: 10.2202/1544-6115.1585.

[34] B. Efron, "Better Bootstrap Confidence Intervals," *Journal of the American Statistical Association*, vol. 82, no. 397, pp. 171–185, 1987, doi: 10.1080/01621459.1987.10478410.

[35] D. R. Cox, "Regression Models and Life-Tables," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 34, no. 2, pp. 187–202, 1972, doi: 10.1111/j.2517-6161.1972.tb00899.x.

Appendix A
Deviations from the Preregistered Plan

Table XI lists every departure from the locked plan with its date, the runs it affected and the action taken; Section IV-I

TABLE XI
DEVIATIONS FROM THE PREREGISTERED PLAN

No.	Date	Deviation	Runs affected	Action
1	2026-09-26	Absolute floor added to the latency rule	3	Floor added; rehearsal runs discarded
2	2026-09-26	Wrong container targeted by a substring match	3	Exact label matching; affected runs collected again
3	2026-09-26	Name mismatch stopped the campaign on a healthy stack	0	Stack check corrected; no data affected
4	2026-09-26	Network-delay rule outlived its run	11	Affected runs collected again; container recreated after each latency run
5	2026-09-27	Two choices the plan left open	0	Settled before the analysis was written
6	2026-09-27	No run censored	0	Rank test made primary for recovery time as the plan required
7	2026-09-27	Asymptotic p-value quoted in the plan	0	Exact value used
8	2026-09-27	Statistical routines vectorised	0	Tested against the reference implementation
9	2026-09-28	Exact p-values computed after the results	0	Reported as supplementary; verdict unchanged
10	2026-10-06	Weight-tuning step not run	0	Inherited weights kept; not re-tuned
11	2026-10-06	Social Network latency drew on all spans	0	Server-only replay; no service-level change
12	2026-10-06	One latency run attempted twice	0	Only the completed attempt analysed
13	2026-10-20	Baseline window includes the load ramp-up	0	Disclosed; baseline gate found no anomalies

summarises them. The repository holds the full record, with a description of each deviation and of the change made.

Dakshrajsinh Tank is a B.Tech student with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. [TO FILL: photograph and biography]

Badrinath N is an Associate Professor with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. [TO FILL: photograph and biography]